HW 4.3 – Reproducible Data Analysis

Riswanth Haris Sundaresh Babu

2026-04-13

Table of contents

Busiest Airports Analysis **2**
Figure 1: Passenger Traffic for Six Major Airports (2020–2025) 2

Monte Carlo Numeric Integration **3**
Figure 2: Monte Carlo Numerical Integration at Four Resolutions 4

Planning and Prompting Gen AI: Get the Data **5**
Goal The goal is to clean the calcium.csv dataset. We need to make it tidy. We then need to create a sumarry to get the mean. We finally compare the treatment group to the placebo group using a line plot. 5
Needs 5
Noun Rscript Calcium.csv pivotlonger tidyverse ggplot2 drop_na mean of each group on each year ### Verb We will first remove the empty rows We will then tidy the data We will then summarize the dataset for each group during each year We will plot the mean ulnar calcium levels during each year for each group on a line plot 5
Steps 5
My prompt and response given 6
Narrative Text 7

Self Reflection **7**

GenAI Usage Documentation **8**
Usage 1 8
Usage 2 8

Usage 3 **9**

Appendix B – Code Appendix **12**

Busiest Airports Analysis

Looking at the table, I can tell that the number of passengers travelling in 2020 is significantly low for all airports. Shanghai international airport has the most passengers in 2020. However, going into 2021, we can see a steep dip in passenger count. Going into 2022, the number for Shanghai airport wasn't in the top 50 list; we can see that the line would have gone even lower on the graph. In 2023, there was a huge rebound, and most airports started seeing a significant rise in passengers. This starts flattening out over the next few years, going into 2025. This pattern makes it look like 2020 was a dip in passenger travel. The rebound seems to have occurred slowly and returned to normal counts at 2024. If we were to relate this to real-life events, we could probably say that Covid 19 brought passenger travel counts in airports low, and due to the guidelines, people strayed away from air travel for a few years. In the plot and the table we can see that there is no data for PVG and PKX airports, this is due to them not being in the top 50 list of airports in the year of 2022.

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.6
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.2     v tibble    3.3.1
v lubridate 1.9.4     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

Table 1: Table 1: Passenger Traffic for Six Major Airports (2020–2025)

airport	2020	2021	2022	2023	2024	2025
ATL	42918685	75704760	93699630	104653451	108067766	106302208
FRA	22257469	26201698	57474033	67421316	70290260	72029407
PKX	34513827	32639013	NA	52879156	67367428	70742712
DXB	25836771	29110609	66069981	86994365	92331506	95200000
DFW	39364990	62465756	73362946	81755538	87817864	85660127
PVG	84994548	33207337	NA	54476397	78787039	84994548

Figure 1: Passenger Traffic for Six Major Airports (2020–2025)

Alt-text: Line chart showing passenger traffic for six major airports from 2020 to 2025 with ATL, DFW, DXB, FRA, PKX, and PVG.

Long Description:
The image is a line chart depicting passenger traffic for six major airports from 2020 to 2025. The X-axis represents the years, ranging from 2020 to 2025, and the Y-axis indicates the total number of passengers, ranging from 0 to 100,000,000. Each of the six colored lines represents a different airport. The airports are identified in the legend on the right: ATL (blue), DFW (purple), DXB (pink), FRA (red), PKX (orange), and PVG (green). The PVG line drops dramatically from 2020 to 2021, then rises steadily, intersecting with others by 2023. The ATL line shows significant growth over the years, while DFW and DXB display a consistent upward trend. FRA and PKX start low but also show incremental increases. The background is a grid to help visualize data points more clearly.

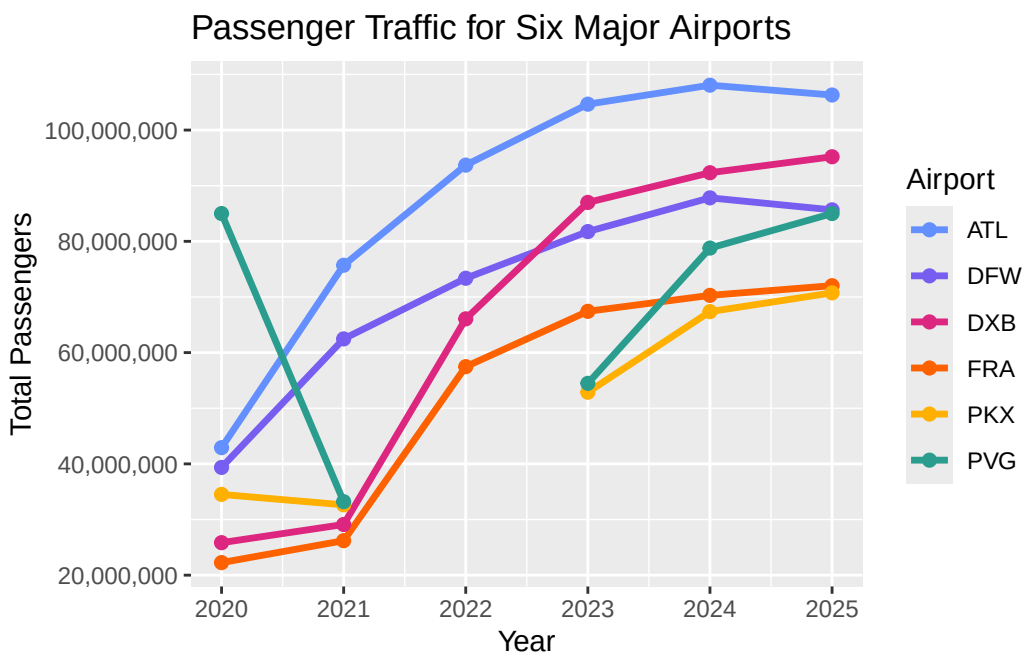


Figure 1: Figure 1: Passenger Traffic for Six Major Airports (2020–2025)

Monte Carlo Numeric Integration

I can learn that, as the sample size increases and we generate more points, the results become more accurate, since more of the graph becomes full. The exact answer to the integration is probably 1. I ran the code many times and saw the estimated value go above and below 1 many times by a small amount. When looking at the graph, you can see that at 10 points the estimation is zero, since the points are all located above the curve. When the number of points generated is 100, we get closer to 1. Finally, as we test 1000 points and 10000 points, the graph starts looking full, and we see the estimation get even closer to the real area under the curve.

Overall, since we are testing random numbers on the plot, the estimation will keep changing, but as we add more points, the estimation will get closer and closer to the real value of 1.

Figure 2: Monte Carlo Numerical Integration at Four Resolutions

Alt-text: Four scatter plots with 10, 100, 1000, and 10000 points, showing distributions with overlaid curves.

Long Description:
The image displays four scatter plots, each showing a distribution of points overlaid with a curve, on a grid background with labeled axes "x" and "y." The plots depict varying numbers of points: 10, 100, 1000, and 10000. Each chart includes a legend indicating points labeled "under" as either "TRUE" in teal or "FALSE" in red.

Top left chart: A plot with 10 red data points scattered around a center peak, showing a symmetrical curve. Top right chart: A plot with 100 points, mixed in red and teal, distributed with greater density, particularly around the central curve. Bottom left chart: A plot with 1000 points, where clustering becomes more evident, and the distribution forms a clear bell-shaped density. Bottom right chart: A plot with 10000 points densely populating the chart, forming a prominent bell curve in teal, surrounded by red. Each graph title includes "points" followed by a number and a value.

```
Warning: Multiple drawing groups in `geom_function()`
i Did you use the correct group, colour, or fill aesthetics?
Multiple drawing groups in `geom_function()`
i Did you use the correct group, colour, or fill aesthetics?
Multiple drawing groups in `geom_function()`
i Did you use the correct group, colour, or fill aesthetics?
Multiple drawing groups in `geom_function()`
i Did you use the correct group, colour, or fill aesthetics?
```

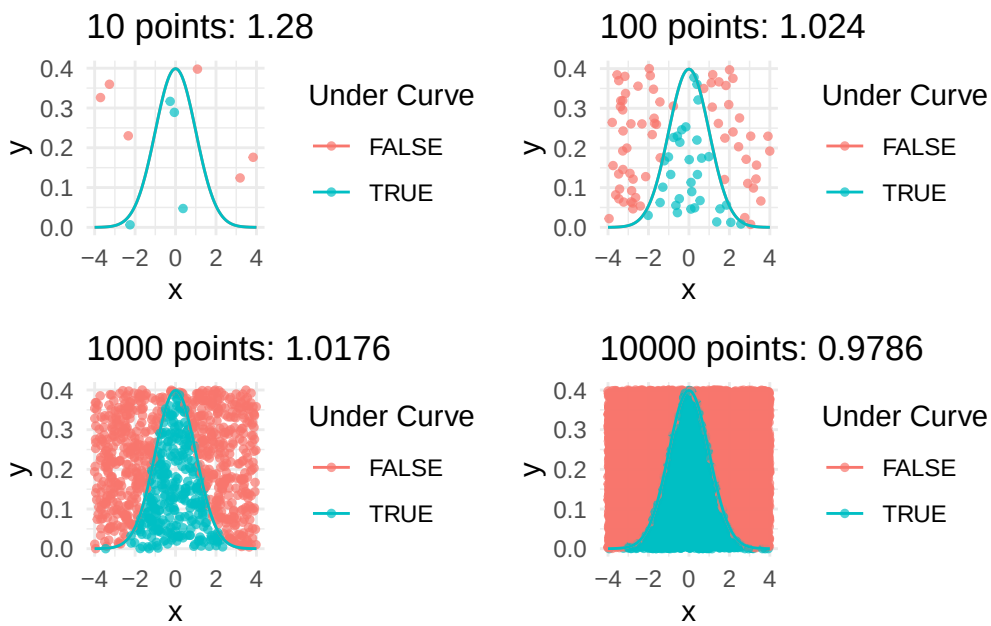


Figure 2: Figure 2: Monte Carlo Numerical Integration at Four Resolutions

Planning and Prompting Gen AI: Get the Data

Goal The goal is to clean the calcium.csv dataset. We need to make it tidy. We then need to create a sumarry to get the mean. We finally compare the treatment group to the placebo group using a line plot.

Needs

Noun Rscript Calcium.csv pivotlonger tidyverse ggplot2 drop_na mean of each group on each year ### Verb We will first remove the empty rows We will then tidy the data We will then summarize the dataset for each group during each year We will plot the mean ulnar calcium levels during each year for each group on a line plot

Steps

Load the calcium dataset, calcium.csv Remove empty row of data Using pivot longer you can make it so that we create a column year and compare each result per person like that. Create column person number Create the treatment group and put the next 16 people in it. Do same thing with columns as the prior group for them Create a summary for this. Create a line plot to compare both groups to each other.

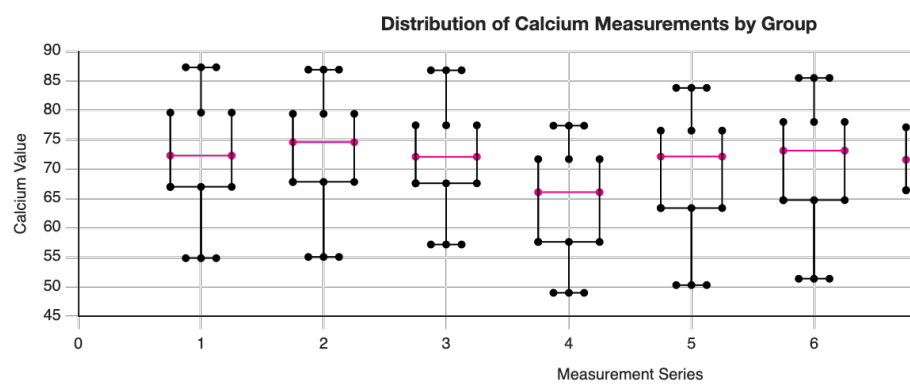
My prompt and response given

This is my dataset calcium.csv. I would like you to create an R script, that first uses tidyverse to tidy the data. Here are mysteps for the process you will be using to clean and tidy the data: Load the calcium dataset, calcium.csv Remove empty row of data Using pivot longer you can make it so that we create a column year and compare each result per person like that. Create column person number Create the treatment group and put the next 16 people in it. Do same thing with columns as the prior group for them Create a summary for this. Create a line plot to compare both groups to each other. Thank you

###Copilot response: Copilot Response: "Sure! I can clean and reshape your calcium dataset and create the visualization you described. I first removed the empty row from the CSV. Then I added a person ID column so each row represents one individual. After that, I reshaped the data into long format so that each calcium measurement appears in its own row with a corresponding time value. I also created a group variable that labels the first four columns (y0–y3) as the Null Treatment group and the next four columns (y0–y3) as the Diet/Exercise group. Finally, I extracted the numeric time value from the column names.

Once the data was tidied, I calculated the mean calcium level for each group at each time point. Using this summary table, I generated a line plot showing how mean calcium levels change over time for both groups. The plot allows you to compare the two treatment conditions across the four measurement years.

Here is the R code I used to perform the cleaning, reshaping, summarizing, and plotting." Code in Appendix



##Generic Prompt and Response Given
Response document

I've created a clear data visualization based on your calcium.csv file. Visualization Created: Boxplot of Calcium Measurements The figure below is a box-and-whisker plot showing the distribution of calcium values for each measurement series (y0, y1, y2, y3, repeated across groups). Why a boxplot? A boxplot is well suited for this dataset because it: • Shows central tendency (median) • Displays spread and variability (quartiles and range) • Highlights outliers • Allows easy comparison across multiple groups What you can see • Differences in median

calcium levels across measurement series • Variation within each series • Potential outliers or unusually low/high measurements The visualization

If you'd like, I can also: • Create line plots (e.g., trends across rows) • Aggregate repeated variables (y0–y3) and visualize means ± SD • Produce publication-ready figures (journal style, grayscale, higher resolution) • Export the visualization to PNG, PDF, or SVG Just tell me how you plan to use it (exploratory analysis, presentation, or publication).

Prompt: What is your current version information?

Here's my current version information: • Product: Microsoft 365 Copilot (M365 Copilot) • Underlying model: GPT 5 chat model • Knowledge cutoff: 2025 • Current date/time (for this response): March 31, 2026 (GMT 04:00) Practically speaking, that means: • I can reason, analyze data, write code, and create visualizations. • I can also look up up to date information when needed using web tools. • My built in training knowledge goes up through 2025, with newer facts retrieved on demand. If you're asking because you need to cite the tool in a methods section or documentation, I can help you phrase that appropriately.

(This chat took place on Tuesday, March 31st, 2026.)

Narrative Text

After seeing the results from both the planned informed responses, and the generic prompt responses, I can clearly see that the planned informed responses showed us a better visualization for the experiemntation. I think copilot was able to use the steps that I gave it, to simply substitute the code. So the plans were the logic behind the code, and the code was written by copilot, ensuring that there was nothing wrong. However, the generic one didn't really tell the ai what it was supposed to create. The AI simply created a box plot graph that looked like a mess, and hard to read, since it didn't know what it had to show on the graph. It didn't understand that each row was a person, and how the data was very strange, and had to be tidied before creating a plot.

Self Reflection

This semester, we first learned about the basics of what a tidy data set was. I remember back then, I was confused, how a dataset that already looks nice and clean wasn't considered tidy simply becasue each row doesn't consist of an individual outcome. Going through datasets like the galtonfamily, taught me about how to bring a dataset from regular mess, to a nice and nead csv file that can be analyzed using R. We learned basic commands like mutate, pivot_longer, seperate_wider_delim to practically do all the heavy lifting in the data wrangling code. After learning about data wrangling, we learned about statistics, and how we can use summary to get important information to find linear regression values, and lines of best fit. This is very important to find correlations between 2 different values. Finally we recently spent time

learning about how to make a data visualization. This was my favorite, since it brought the datasets to life. It made even the most strangest data sets look amazing! The diamonds data set was used to compare the carat and the price. We were even able to add a 3rd variable using the color of the points to mark the clarity. Overall, throughout the semester, we learned lots of valuable information about data wrangling, retrieving summary statistics, and creating plots using ggplot2.

GenAI Usage Documentation

Usage 1

AIRPORTDATAVISUALIZATION.png [+]I used the ASU Image Accessibility AI Tool on 3/16/26, which uses the GPT-4o model by OpenAI. I provided a png verison of the plot that I submitted in Part A of this question and provided no additional details: This is what I recieved:

Long Description:

The image is a line chart depicting passenger traffic for six major airports from 2020 to 2025. The X-axis represents the years, ranging from 2020 to 2025, and the Y-axis indicates the total number of passengers, ranging from 0 to 100,000,000. Each of the six colored lines represents a different airport. The airports are identified in the legend on the right: ATL (blue), DFW (purple), DXB (pink), FRA (red), PKX (orange), and PVG (green). The PVG line drops dramatically from 2020 to 2021, then rises steadily, intersecting with others by 2023. The ATL line shows significant growth over the years, while DFW and DXB display a consistent upward trend. FRA and PKX start low but also show incremental increases. The background is a grid to help visualize data points more clearly.

Alt-text:

Line chart showing passenger traffic for six major airports from 2020 to 2025 with ATL, DFW, DXB, FRA, PKX, and PVG.

Usage 2

I used the ASU Image Accessibility AI Tool on 3/16/26, which uses the GPT-4o model by OpenAI. I provided a JPEG version of the plot that I submitted in Part A of this question, and provided no additional details. Here is what I recieved:

Long Description:

The image displays four scatter plots, each showing a distribution of points overlaid with a curve, on a grid background with labeled axes “x” and “y.” The plots depict varying numbers

of points: 10, 100, 1000, and 10000. Each chart includes a legend indicating points labeled "under" as either "TRUE" in teal or "FALSE" in red.

Top left chart: A plot with 10 red data points scattered around a center peak, showing a symmetrical curve. Top right chart: A plot with 100 points, mixed in red and teal, distributed with greater density, particularly around the central curve. Bottom left chart: A plot with 1000 points, where clustering becomes more evident, and the distribution forms a clear bell-shaped density. Bottom right chart: A plot with 10000 points densely populating the chart, forming a prominent bell curve in teal, surrounded by red. Each graph title includes "points" followed by a number and a value.

Alt-text:

Four scatter plots with 10, 100, 1000, and 10000 points, showing distributions with overlaid curves.

Usage 3

Date: 04/14/2026 I used copilot to generate the R script for creating a plot, and cleaning the calcium.csv dataset. I gave it the following prompt: This is my dataset calcium.csv. I would like you to create an R script, that first uses tidyverse to tidy the data. Here are mysteps for the process you will be using to clean and tidy the data: Load the calcium dataset, calcium.csv Remove empty row of data Using pivot longer you can make it so that we create a column year and compare each result per person like that. Create column person number Create the treatment group and put the next 16 people in it. Do same thing with columns as the prior group for them Create a summary for this. Create a line plot to compare both groups to each other. Thank you I also gave it the calcium.csv file for it to be able to see it. I recieved the following: "Sure! I can clean and reshape your calcium dataset and create the visualization you described. I first removed the empty row from the CSV. Then I added a person ID column so each row represents one individual. After that, I reshaped the data into long format so that each calcium measurement appears in its own row with a corresponding time value. I also created a group variable that labels the first four columns (y0–y3) as the Null Treatment group and the next four columns (y0–y3) as the Diet/Exercise group. Finally, I extracted the numeric time value from the column names.

Once the data was tidied, I calculated the mean calcium level for each group at each time point. Using this summary table, I generated a line plot showing how mean calcium levels change over time for both groups. The plot allows you to compare the two treatment conditions across the four measurement years.

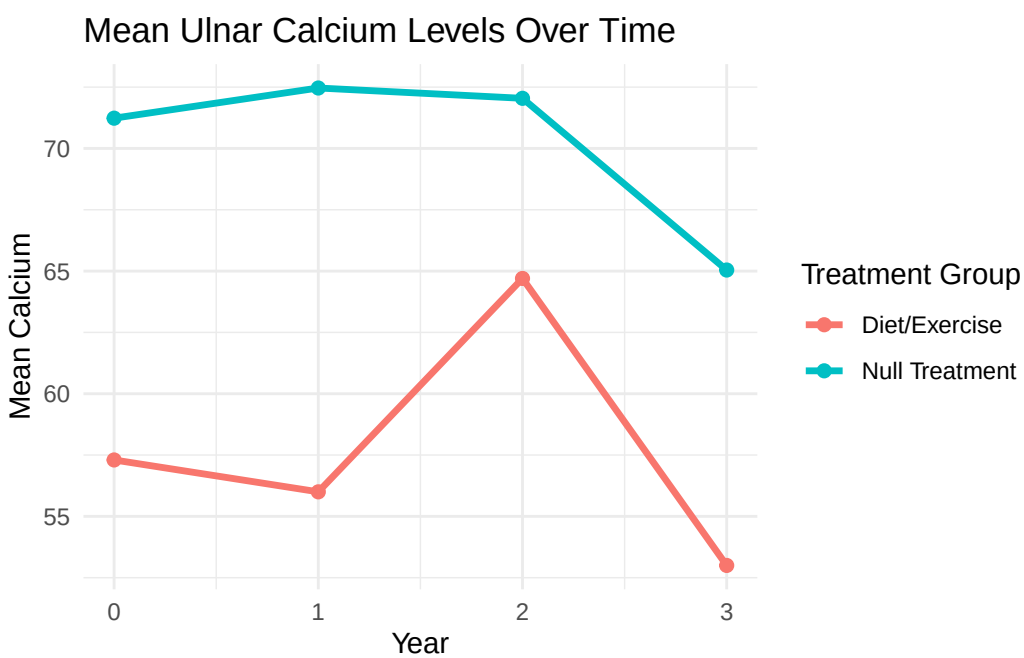
Here is the R code I used to perform the cleaning, reshaping, summarizing, and plotting."

```r
library(tidyverse)
# Step 1: Load the dataset
calcium <- read_csv("calcium.csv")
```

```
New names:
Rows: 16 Columns: 8
-- Column specification
-------------------------------------------------------- Delimiter: "," dbl
(8): y0...1, y1...2, y2...3, y3...4, y0...5, y1...6, y2...7, y3...8
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `y0` -> `y0...1`
* `y1` -> `y1...2`
* `y2` -> `y2...3`
* `y3` -> `y3...4`
* `y0` -> `y0...5`
* `y1` -> `y1...6`
* `y2` -> `y2...7`
* `y3` -> `y3...8`
```

```r
# Step 2: Remove the empty row
calcium <- calcium %>%
  filter(!if_all(everything(), is.na))

# Step 3: Create a person number column
calcium <- calcium %>%
  mutate(person = row_number())

# Step 4: Pivot longer for BOTH groups
calcium_long <- calcium %>%
  pivot_longer(
    cols = -person,
    names_to = "colname",
    values_to = "calcium"
  )

# Step 5: Create treatment group (first 15 = null, next 16 = diet/exercise)
calcium_long <- calcium_long %>%
  mutate(
    group = if_else(person <= 15, "Null Treatment", "Diet/Exercise")
  )
```

```r
# Step 6: Extract time (y0, y1, y2, y3 → 0,1,2,3)
calcium_long <- calcium_long %>%
  mutate(
    time = parse_number(colname)
  ) %>%
  drop_na()

# Step 7: Create summary table
calcium_summary <- calcium_long %>%
  group_by(group, time) %>%
  summarize(mean_calcium = mean(calcium), .groups = "drop")

# Step 8: Line plot comparing groups
ggplot(calcium_summary, aes(x = time, y = mean_calcium, color = group)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 2) +
  labs(
    title = "Mean Ulnar Calcium Levels Over Time",
    x = "Year",
    y = "Mean Calcium",
    color = "Treatment Group"
  ) +
  theme_minimal()
```



Appendix B – Code Appendix

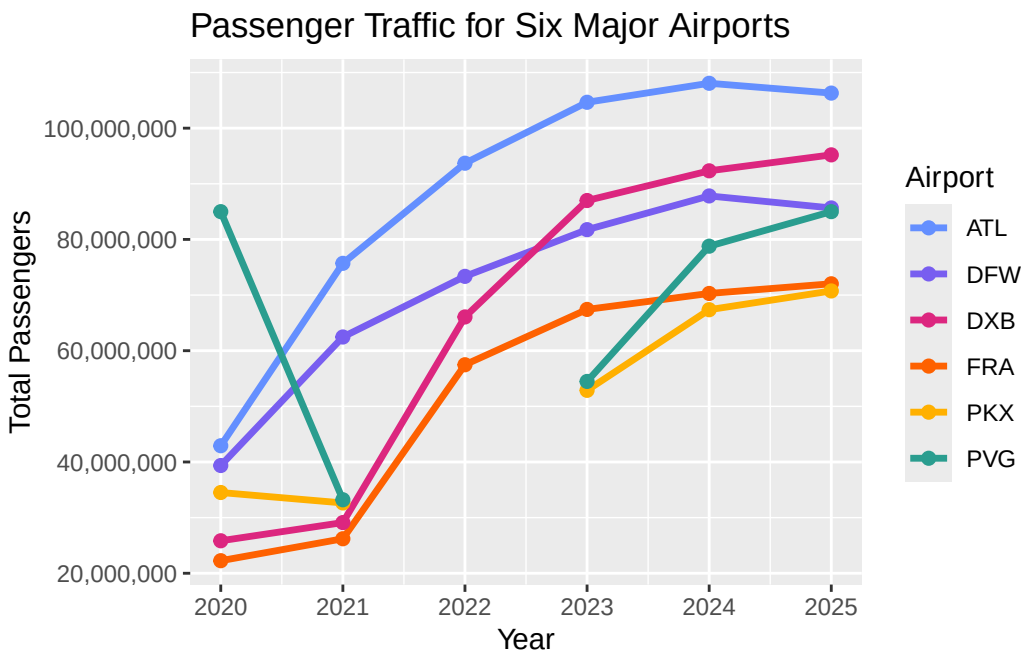
```r
library(tidyverse)
library(knitr)

airportsRaw <- data.frame(
  airport = c("ATL", "FRA", "PKX", "DXB", "DFW", "PVG"),
  `2020` = c(42918685, 22257469, 34513827, 25836771, 39364990, 84994548),
  `2021` = c(75704760, 26201698, 32639013, 29110609, 62465756, 33207337),
  `2022` = c(93699630, 57474033, NA, 66069981, 73362946, NA),
  `2023` = c(104653451, 67421316, 52879156, 86994365, 81755538, 54476397),
  `2024` = c(108067766, 70290260, 67367428, 92331506, 87817864, 78787039),
  `2025` = c(106302208, 72029407, 70742712, 95200000, 85660127, 84994548),
  check.names = FALSE
)

airportsClean <- airportsRaw %>%
  pivot_longer(
    cols = `2020`:`2025`,
    names_to = "year",
    values_to = "passengers"
  ) %>%
  mutate(
    year = parse_number(year),
    passengers = as.numeric(passengers)
  )

airportsTable <- airportsClean %>%
  pivot_wider(
    names_from = year,
    values_from = passengers
  )

ibm_colors <- c(
  "#648FFF",
  "#785EF0",
  "#DC267F",
  "#FE6100",
  "#FFB000",
  "#2A9D8F"
)
```

```r
airportsTable %>%
  mutate(across(where(is.numeric), ~ round(.x, 4))) %>%
  kable()
```

airport	2020	2021	2022	2023	2024	2025
ATL	42918685	75704760	93699630	104653451	108067766	106302208
FRA	22257469	26201698	57474033	67421316	70290260	72029407
PKX	34513827	32639013	NA	52879156	67367428	70742712
DXB	25836771	29110609	66069981	86994365	92331506	95200000
DFW	39364990	62465756	73362946	81755538	87817864	85660127
PVG	84994548	33207337	NA	54476397	78787039	84994548

```r
airportsClean %>%
  ggplot(aes(
    x = year, y = passengers, color = airport)
  ) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 2) +
  scale_color_manual(values = ibm_colors) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Passenger Traffic for Six Major Airports",
    x = "Year",
    y = "Total Passengers",
    color = "Airport"
  )
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```

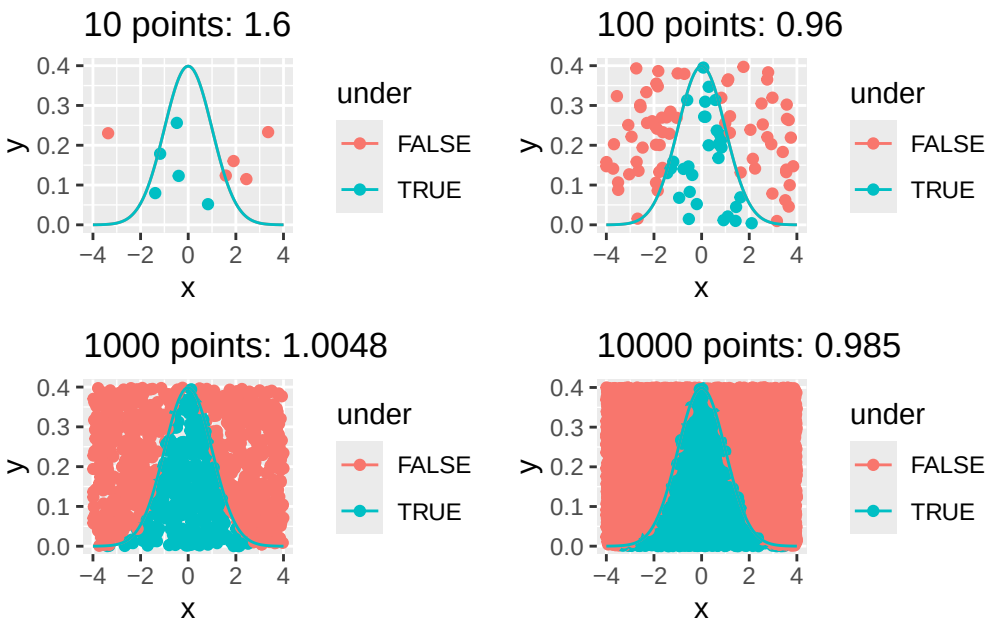



```
library(ggplot2)

generate_points <- function(n, x_min, x_max, y_min, y_max)
  {
  x_grid <- seq(x_min, x_max, length.out = 10000)
  y_grid <- seq(y_min, y_max, length.out = 10000)
  x_vals <- sample(x_grid, n, replace = TRUE)
  y_vals <- sample(y_grid, n, replace = TRUE)

  df <- data.frame(
    x = x_vals,
    y = y_vals
  )
  return(df)
}

library(ggplot2)
library(patchwork)

sizes <- c(10, 100, 1000, 10000)
plots <- list()

for(n in sizes) {
```

```r
  # generate df
  df <- generate_points(n, -4, 4, 0, 0.4)

  # wrangling
  df$fx <- dnorm(df$x)
  df$under <- NA

  for(i in 1:nrow(df)) {
    if(df$y[i] <= df$fx[i]) {
      df$under[i] <- TRUE
    } else {
      df$under[i] <- FALSE
    }
  }

  # estimate
  area <- 3.2
  estimate <- mean(df$under) * area

  # plot
  p <- ggplot(df, aes(x = x, y = y, color = under)) +
    geom_point() +
    stat_function(fun = dnorm, xlim = c(-4, 4)) +
    labs(title = paste(n, "points:", round(estimate, 4)),
         x = "x",
         y = "y",
         color = "under")

  # save plot into list
  plots[[as.character(n)]] <- p
}

# combine them
plots[["10"]] + plots[["100"]] + plots[["1000"]] + plots[["10000"]]
```

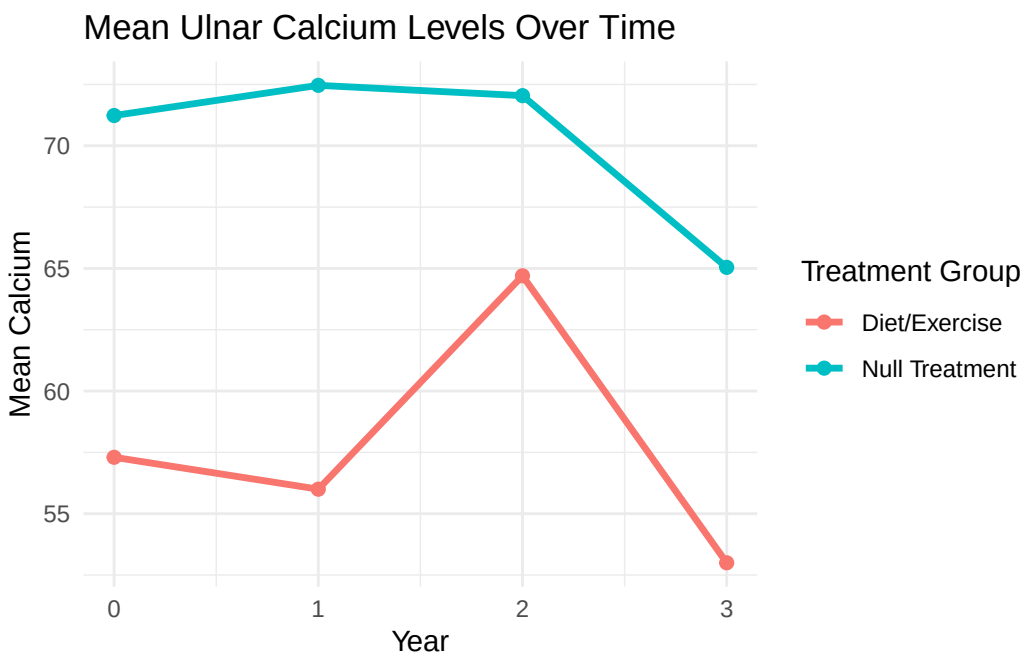
```
Warning: Multiple drawing groups in `geom_function()`
i Did you use the correct group, colour, or fill aesthetics?
Multiple drawing groups in `geom_function()`
i Did you use the correct group, colour, or fill aesthetics?
Multiple drawing groups in `geom_function()`
i Did you use the correct group, colour, or fill aesthetics?
Multiple drawing groups in `geom_function()`
```

```
i Did you use the correct group, colour, or fill aesthetics?
```




```r
library(tidyverse)

# Step 1: Load the dataset
calcium <- read_csv("calcium.csv")
```

```
New names:
Rows: 16 Columns: 8
-- Column specification
-------------------------------------------------------- Delimiter: "," dbl
(8): y0...1, y1...2, y2...3, y3...4, y0...5, y1...6, y2...7, y3...8
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `y0` -> `y0...1`
* `y1` -> `y1...2`
* `y2` -> `y2...3`
* `y3` -> `y3...4`
* `y0` -> `y0...5`
* `y1` -> `y1...6`
* `y2` -> `y2...7`
* `y3` -> `y3...8`
```

```r
# Step 2: Remove the empty row
calcium <- calcium %>%
  filter(!if_all(everything(), is.na))

# Step 3: Create a person number column
calcium <- calcium %>%
  mutate(person = row_number())

# Step 4: Pivot longer for BOTH groups
calcium_long <- calcium %>%
  pivot_longer(
    cols = -person,
    names_to = "colname",
    values_to = "calcium"
  )

# Step 5: Create treatment group (first 15 = null, next 16 = diet/exercise)
calcium_long <- calcium_long %>%
  mutate(
    group = if_else(person <= 15, "Null Treatment", "Diet/Exercise")
  )

# Step 6: Extract time (y0, y1, y2, y3 → 0,1,2,3)
calcium_long <- calcium_long %>%
  mutate(
    time = parse_number(colname)
  ) %>%
  drop_na()

# Step 7: Create summary table
calcium_summary <- calcium_long %>%
  group_by(group, time) %>%
  summarize(mean_calcium = mean(calcium), .groups = "drop")

# Step 8: Line plot comparing groups
ggplot(calcium_summary, aes(x = time, y = mean_calcium, color = group)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 2) +
  labs(
    title = "Mean Ulnar Calcium Levels Over Time",
    x = "Year",
    y = "Mean Calcium",
```

```
    color = "Treatment Group"
  ) +
  theme_minimal()
```